Day 0 - Before You Write Any Code: The Absolute Basics

Day 0 — Before You Write Any Code: The Absolute Basics

This day exists so that Day 1 doesn't feel overwhelming. If you've already worked through the Python track's Day 0 primer, some of this will repeat (what a terminal is, what an error message means) — that's intentional, because this lesson is written to stand completely on its own. If any part is already familiar, skim it and move on.

What is JavaScript, and why is it unusual among programming languages?

JavaScript (often abbreviated **JS**) is a programming language — a way of writing instructions that a computer follows exactly, step by step (see the Python primer's "what is a program" section for the full version of this idea, if you want it — the short version: code is a list of instructions, and the computer does exactly what it says, not what you meant).

What makes JavaScript unusual is *where* it runs. Most languages run in one place — a program you install and execute. JavaScript runs in **two** very different places:

1. **Inside a web browser** (Chrome, Firefox, Edge, Safari) — this is JavaScript's original home. Every web page you've ever visited that does something interactive (a button that reacts when clicked, a form that checks your input before submitting, content that updates without reloading the page) is running JavaScript, right there in your browser, on your own computer.
2. **On a server, or directly on your computer, outside any browser at all** — using a program called **Node.js** (or just "Node"), which lets you run JavaScript exactly like you'd run a Python script, from your terminal, with no browser involved whatsoever.

This course teaches you the JavaScript *language* itself first, running it with Node — the exact same language rules apply whether the code eventually ends up running in a browser or in Node, so everything you learn transfers directly. You'll return to browser-specific JavaScript (interacting with a web page) properly if/when you study the HTML & CSS track's more advanced material, since that requires knowing HTML and CSS first.

What is TypeScript, and how does it relate to JavaScript?

TypeScript (often abbreviated **TS**) is not a separate, competing language — it's JavaScript, plus an extra layer of optional rules that let you describe what *type* of value each variable and function is expected to hold (a number, a piece of text, and so on), which a tool can then check *before* your code ever runs, catching a whole category of mistakes early. You'll spend Week 1 and most of Week 2 (Days 1-10) learning plain JavaScript, since TypeScript is built entirely on top of it and makes no sense without it — then Days 11-14 introduce TypeScript itself and have you use it for real.

Text editors and the terminal — same as any language

You'll write code in a plain-text code editor (VS Code is the most common choice, and happens to be made by the same company that created TypeScript, so it has excellent built-in support for both languages). If you already set up a terminal workflow for the Python track, you can reuse the exact same terminal window and editor here — nothing about your basic tools changes between languages. If this is genuinely your first time, see this same section in the Python track's `day00-primer/lesson.md` for a full explanation of what a text editor and a terminal actually are, since that explanation applies identically here.

A JavaScript file is a plain text file ending in `.js` (or `.ts` for TypeScript, starting Day 11) — for example, `hello.js`.

Running a JavaScript file with Node.js

Once you've saved a file called `hello.js` containing:

```
console.log("Hello, world!");
```

you run it from the terminal, in the same folder as the file, by typing:

```
node hello.js
```

Node reads `hello.js` top to bottom and runs each instruction. Here, that prints `Hello, world!` to the terminal.

`console.log(...)` is JavaScript's version of Python's `print(...)` — a built-in instruction (a "function," fully explained on Day 4) that displays whatever is inside its parentheses. You will use it constantly, especially early on, to see exactly what your code is doing at each step.

Notice the semicolon `;` at the end of that line. JavaScript uses semicolons to mark the end of a statement (roughly, "one complete instruction"), similar to how a period ends a sentence in English. JavaScript will often still work even if you forget one, thanks to a feature called "automatic semicolon insertion" that tries to guess where you meant to put them — but relying on that guessing is a common source of subtle, confusing bugs. **This course writes semicolons explicitly at the end of every statement, and you should too.**

The Node.js REPL — JavaScript's interactive shell

Just like Python has an interactive mode, typing `node` alone (with nothing after it) and pressing Enter drops you into Node's **REPL** (Read-Evaluate-Print Loop — see the Python primer for the full explanation of what this acronym means if you want it) — a mode where you type one line of JavaScript at a time and immediately see the result:

```
> 2 + 2
4
> console.log("hi")
hi
```

The `>` is Node's prompt — you don't type it yourself. To leave, type `.exit` and press Enter, or press Ctrl+C twice.

The browser console — JavaScript's OTHER interactive shell

Because JavaScript also runs inside web browsers, every modern browser has its own built-in REPL-like tool, called the **console**, part of a larger toolset called **DevTools** (Developer Tools). You open it by pressing F12 (or right-clicking a web page and choosing "Inspect"), then clicking the "Console" tab. You can type JavaScript directly in there too, and it runs immediately, exactly like Node's REPL — except it also has access to whatever web page is currently open. You won't need this until later, once you start working with actual web pages, but it's worth knowing this second REPL exists, since you'll see screenshots and instructions referencing it constantly in any JavaScript material you read online.

Reading a JavaScript error message

You will see error messages constantly — this is normal and expected, not a sign that you've broken something badly. Here's an example:

```
console.log("Hello"
```

Running this (notice the missing closing parenthesis) produces something like:

```
SyntaxError: missing ) after argument list
```

Just like Python, the error's category name (`SyntaxError`, and later you'll meet `TypeError`, `ReferenceError`, and others) tells you something specific: a `SyntaxError` means the problem is in how you *typed* the code — punctuation, structure — before the code even had a chance to run. A `TypeError` (which you'll meet properly Day 1) means the code ran, but tried to do something with a value that doesn't support that operation. Node will usually also show you a "stack trace" — several lines showing which file and line number the error happened at, and what called what, leading up to it. Just like the Python advice: **read the error message fully, starting with its category name and description, before touching your code.** Don't panic and start randomly changing things.

Curly braces `{ }` — how JavaScript groups instructions together

Unlike Python, which uses indentation alone to show which lines belong together (see the Python primer if you're curious), JavaScript uses **curly braces** `{ }` to explicitly mark the start and end of a block of code that belongs together:

```
if (5 > 3) {
    console.log("five is bigger");   // this line is INSIDE the if block
    console.log("still inside");      // also inside
}
console.log("this always runs");        // OUTSIDE the if block -- always runs
```

Indentation (the spaces at the start of a line) is still used in JavaScript, but purely for *human* readability — unlike Python, changing the indentation alone, without touching the curly braces, does not change what the code actually does. The curly braces are what genuinely matter to JavaScript; the indentation is a courtesy to whoever reads the code (including future-you), and every editor will auto-indent for you as you type, matching the braces.

Comments — notes to humans that JavaScript ignores

```
// this is a single-line comment -- everything after // on this line is ignored

/* this is a
   multi-line comment --
   everything between /* and */ is ignored */
```

Exactly like Python's `#`, comments exist purely for a human reader — JavaScript skips them entirely when running your code.

A few words you'll see constantly, defined up front

- **Variable** — a name that refers to a value (full explanation Day 1).
- **Function** — a named, reusable block of instructions you can "call" (run) by name, optionally handing it some input (full explanation Day 4). `console.log(...)` is a function.
- **Argument** — a value you hand to a function when you call it. In `console.log("hi")`, `"hi"` is the argument.
- **String** — text data, written between quotes: `"hello"`, `'hello'`, or, as you'll see Day 5, using backticks: `` `hello` ``.
- **Object** — in JavaScript, this word is used two ways, and it's worth flagging now so it doesn't confuse you later: in the general sense (like Python), it means "a piece of data" broadly. In the *specific* sense you'll meet on Day 3, `Object` refers to one particular kind of JavaScript value — a collection of named properties, similar to a Python dict. Context will make clear which meaning is intended.
- **Method** — a function that belongs to a specific piece of data, called with a dot, like `"hello".toUpperCase()`.
- **Bug** — a mistake in your code that makes it behave incorrectly. Not a moral failing — every programmer produces bugs constantly.
- **Debugging** — the process of finding and fixing a bug.

What "run the file and check the output" means for these exercises

Starting Day 1, every `exercises.js` file has spots marked `// TODO: implement`, similar in spirit to Python's `pass` placeholder — JavaScript's equivalent of "do nothing yet" is simply to leave a function body empty, or return `undefined` (JavaScript's version of Python's `None`, meaning "no value" — full explanation Day 1). Below the TODOs, each file has code that calls your functions with test inputs and prints whether each one is correct. Your loop for every day: write code → run `node exercises.js` → read the output → fix what's wrong → run again. This is, in essence, the daily rhythm of a working programmer at any experience level.

You're ready for Day 1

Go to `day01-basics/lesson.md` next.